HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING
FACULTY OF INTERNATIONAL EDUCATION

ò&ô

EMBEDDED SYSTEM DESIGN COMPUTER ENGINEERING

TOPIC

DESIGN AND IMPLEMENTATION OF AN STM32-BASED IoT WEATHER STATION USING ESP32 AND MQTT

Student : **ĐẶNG NGUYỄN NAM NHẬT**

NGÔ PHÚC KHANG

MSSV : **22119022**

20119117

Supervisor : **TEACHER HUYNH HOANG HA**

TP. HỒ CHÍ MINH 10 /2026

PROJECT ASSIGNMENT

Project title: Design and Implementation of an STM32-Based IoT Weather Station Using ESP32 and MQTT.

The project develops a compact embedded weather monitoring system in which the STM32F103C8T6 is used as the main controller for sensor acquisition and local processing. An ESP32 is used as a communication gateway to provide Wi-Fi connectivity, MQTT publishing, and remote monitoring through Node-RED.

Main Tasks

- Study the operating principles of the STM32F103C8T6, BMP388, SHT30, BH1750, rain sensor, OLED display, buzzer, and ESP32.
- Design and implement the hardware connections for environmental sensing and local display.
- Develop STM32 firmware to read temperature, humidity, pressure, light intensity, and rain level.
- Classify rain status into DRY, LOW, and HEAVY states and provide buzzer warnings.
- Transmit processed weather data from STM32 to ESP32 through UART.
- Connect the ESP32 to Wi-Fi and publish sensor data to an MQTT broker.
- Build a Node-RED flow and dashboard to visualize real-time weather information.
- Test sensor operation, rain warning, Wi-Fi/MQTT reconnection, and power-cycle recovery.

Deliverables

- Working STM32 IoT Weather Station hardware prototype.
- STM32 firmware and ESP32 PlatformIO project.
- MQTT communication and Node-RED dashboard.
- Technical report and testing evidence.

ACKNOWLEDGEMENTS

During the implementation of this Embedded System Design project, the author received valuable guidance and support from lecturers, classmates, friends, and family. The knowledge acquired from embedded programming, digital interfaces, microcontroller systems, and IoT communication was essential for completing the project.

The author would like to express sincere appreciation to the project supervisor for providing technical direction, reviewing the system design, and giving feedback during the development process. These suggestions helped identify suitable hardware, improve the firmware structure, and resolve integration problems between the STM32, sensors, ESP32, MQTT broker, and dashboard.

The author also thanks the manufacturers and open-source communities that provide datasheets, device drivers, development tools, and documentation used during the project. Although the system has been tested successfully, some limitations remain and are discussed in the final chapter.

ABSTRACT

Weather monitoring is a common application of embedded systems and Internet of Things (IoT) technologies because it combines sensor acquisition, local processing, communication, and remote visualization in one complete system. This project presents the design and implementation of an STM32-based IoT Weather Station intended as a compact academic prototype for monitoring basic environmental conditions.

The STM32F103C8T6 operates as the main controller. A SHT30 sensor is used for temperature and humidity, a BMP388 sensor is used for atmospheric pressure, a BH1750 sensor is used for ambient light, and an analog rain sensor is used to estimate surface wetness. The digital sensors share the I2C1 bus, while the rain sensor is sampled through the STM32 ADC. The processed values are displayed locally on a 0.96-inch OLED. The rain ADC value is classified into DRY, LOW RAIN, and HEAVY RAIN states using experimentally selected thresholds, and an active buzzer provides different warning behavior according to the rain state.

For IoT connectivity, the STM32 transmits processed weather data to an ESP32 through UART at 115200 bit/s. The ESP32 acts as a communication gateway rather than the primary controller. It connects to Wi-Fi, publishes the received data to a HiveMQ MQTT broker, and automatically attempts to restore Wi-Fi and MQTT connections after an interruption. Node-RED subscribes to the MQTT topics and displays temperature, humidity, pressure, light, rain ADC, rain state, device status, and temperature history on a web dashboard.

The final prototype was tested through normal operation, rain-state changes, buzzer activation, Wi-Fi loss, MQTT reconnection, and complete power cycling. The system recovered automatically after connectivity and power interruptions and continued publishing data without manual reconfiguration. The project therefore demonstrates a complete end-to-end embedded IoT architecture from sensing and local control to wireless communication and remote visualization.

Keywords: STM32F103C8T6, ESP32, IoT, Weather Station, MQTT, Node-RED, SHT30, BMP388, BH1750.

TABLE OF CONTENTS

PROJECT ASSIGNMENT **2**
Main Tasks 2
Deliverables 2
ACKNOWLEDGEMENTS` **3**
ABSTRACT **4**
TABLE OF CONTENTS **5**
LIST OF ABBREVIATIONS **10**
LIST OF FIGURES **11**
LIST OF TABLES **12**
CHAPTER 1: INTRODUCTION **13**
1.1 PROJECT INTRODUCTION 13
1.2 MOTIVATION OF THE PROJECT 13
1.3 PROJECT OBJECTIVES 13
1.4 RESEARCH SUBJECTS 14
1.5 SCOPE OF THE PROJECT 14
1.6 RESEARCH METHODOLOGY 14
1.7 REPORT ORGANIZATION 14
CHAPTER 2: LITERATURE REVIEW **15**
2.1 OVERVIEW OF IoT WEATHER MONITORING SYSTEMS 15
2.2 STM32F103C8T6 MICROCONTROLLER 15
2.3 BMP388 PRESSURE SENSOR 15
2.4 SHT30 TEMPERATURE AND HUMIDITY SENSOR 16
2.5 BH1750 LIGHT SENSOR 16
2.6 ANALOG RAIN SENSOR 16
2.7 OLED DISPLAY 17

2.8 ACTIVE BUZZER 17
2.9 ESP32 Wi-Fi GATEWAY 17
2.10 COMMUNICATION INTERFACES 17
2.10.1 I2C 17
2.10.2 ADC 17
2.10.3 UART 18
2.11 MQTT, HIVEMQ AND NODE-RED 18
2.12 SOFTWARE DEVELOPMENT ENVIRONMENT 18
2.13 CHAPTER SUMMARY 18
CHAPTER 3: SYSTEM DESIGN 19
3.1 SYSTEM REQUIREMENTS 19
3.2 OVERALL SYSTEM ARCHITECTURE 19
3.3 HARDWARE CONNECTION DESIGN 20
3.4 SENSOR DATA ACQUISITION AND PROCESSING 20
3.5 RAIN CLASSIFICATION AND BUZZER LOGIC 20
3.6 UART WEATHER DATA FRAME 20
3.7 ESP32 Wi-Fi AND MQTT GATEWAY 21
3.8 MQTT TOPIC ORGANIZATION 21
3.9 NODE-RED DASHBOARD DESIGN 21
3.10 SYSTEM OPERATING FLOWCHART 22
3.11 SOFTWARE STRUCTURE AND RESPONSIBILITIES 22
CHAPTER 4: SYSTEM IMPLEMENTATION 23
4.1 HARDWARE IMPLEMENTATION 23
4.2 STM32 FIRMWARE IMPLEMENTATION 24
4.3 ESP32 FIRMWARE IMPLEMENTATION 24
4.4 MQTT BROKER IMPLEMENTATION 24
4.5 NODE-RED FLOW IMPLEMENTATION 25
4.6 DASHBOARD IMPLEMENTATION 25
4.7 STARTUP, RECONNECTION, AND FAILURE RECOVERY 25

CHAPTER 5: RESULTS AND SYSTEM EVALUATION 26
5.1 SYSTEM STARTUP AND SENSOR OPERATION 26
5.2 LOCAL OLED DISPLAY RESULT 26
5.3 STM32-TO-ESP32, Wi-Fi AND MQTT RESULT 27
5.4 MQTT BROKER RESULT 29
5.5 NODE-RED DASHBOARD RESULT 29
5.6 RAIN SENSOR AND BUZZER TEST 30
5.7 Wi-Fi AND MQTT RECONNECTION TEST 30
5.8 POWER-CYCLE TEST 30
5.9 SYSTEM EVALUATION 31
5.10 EXISTING LIMITATIONS 31
CHAPTER 6: CONCLUSION AND FUTURE WORK 31
6.1 CONCLUSION 31
6.2 FUTURE WORK 32
REFERENCES 33
APPENDIX A: FINAL PIN CONNECTION SUMMARY 34
APPENDIX B: MQTT TOPIC LIST 34
APPENDIX C: FINAL FUNCTIONAL CHECKLIST 34
APPENDIX D: DEMONSTRATION AND TEST PROCEDURE 35

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ADC	Analog-to-Digital Converter
CPU	Central Processing Unit
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
IoT	Internet of Things
MQTT	Message Queuing Telemetry Transport
OLED	Organic Light-Emitting Diode
RX	Receive
SCL	Serial Clock Line
SDA	Serial Data Line
UART	Universal Asynchronous Receiver/Transmitter
TX	Transmit
Wi-Fi	Wireless Fidelity

LIST OF FIGURES

Figure 2.1: STM23F103C8T6
Figure 2.2: BMP388 Sensor
Figure 2.3: SHT30 Sensor
Figure 2.4: BH1750 Sensor
Figure 2.5: Rain local display Sensor
Figure 2.6: OLED 0.96 Display
Figure 2.7: Active Buzzer
Figure 2.8: ESP32 Gateway
Figure 3.1: Overall block diagram of the proposed system
Figure 3.2: Circuit Schematic of the Weather Station System
Figure 3.3: System operating flowchart
Figure 4.1: Final hardware prototype
Figure 4.2: Final Node-RED MQTT flow
Figure 4.3: Dashboard Implemention
Figure 5.1: OLED local display
Figure 5.2: ESP32 serial monitor
Figure 5.3: HiveMQ WebSocket Client
Figure 5.4: Node-RED dashboard

LIST OF TABLES

Table 1: Main hardware components
Table 2: STM32F103C8T6 main specifications
Table 3: Sensor and peripheral connections
Table 4: Rain classification thresholds
Table 5: UART weather data packet
Table 6: MQTT topic organization
Table 7: Functional test results

CHAPTER 1: INTRODUCTION

1.1 PROJECT INTRODUCTION

With the development of embedded systems and Internet of Things technologies, environmental monitoring devices can now collect physical parameters locally and deliver them to remote applications in real time. Weather information such as temperature, humidity, atmospheric pressure, light intensity, and rain status is useful for residential monitoring, laboratory experiments, agricultural applications, and small-scale automation systems.

This project, entitled "Design and Implementation of an STM32-Based IoT Weather Station Using ESP32 and MQTT," develops a complete prototype based on a two-controller architecture. The STM32F103C8T6 performs sensor acquisition, local processing, OLED display control, and buzzer warning. The ESP32 is used as a network gateway that receives processed data through UART, connects to Wi-Fi, and publishes the information to an MQTT broker. Node-RED then subscribes to the MQTT topics and displays the measurements on a dashboard.

Separating the sensing controller from the network controller provides a clear system structure. The STM32 can continue performing local sensing and warning tasks independently of the Internet connection, while the ESP32 focuses on communication and remote monitoring. This architecture also provides a useful learning platform for I2C, ADC, UART, Wi-Fi, MQTT, and dashboard development.

1.2 MOTIVATION OF THE PROJECT

Many simple weather-monitoring projects either display measurements locally without remote access or rely entirely on a Wi-Fi microcontroller to perform sensing and networking. For an embedded-system design course, a more meaningful implementation is to divide the system into dedicated processing and communication functions. This allows the project to demonstrate both low-level microcontroller interfacing and practical IoT connectivity.

The project was therefore selected to provide practical experience with sensor integration, real-time firmware development, data formatting, serial communication, wireless networking, MQTT messaging, and dashboard visualization. The system also demonstrates fault recovery through automatic Wi-Fi/MQTT reconnection and power-cycle testing.

1.3 PROJECT OBJECTIVES

- Design a weather station based on the STM32F103C8T6 as the main embedded controller.
- Measure temperature, humidity, atmospheric pressure, light intensity, and rain level.
- Display the latest values locally on an OLED display.
- Classify rain conditions into DRY, LOW, and HEAVY states.
- Generate a buzzer warning according to the detected rain condition.
- Transmit processed weather data from STM32 to ESP32 through UART.
- Connect the ESP32 to Wi-Fi and publish data using MQTT.
- Use HiveMQ as the MQTT broker and Node-RED as the visualization platform.
- Implement and verify Wi-Fi/MQTT reconnection and system restart behavior.

1.4 RESEARCH SUBJECTS

Hardware used in the project includes the STM32F103C8T6 development board, ESP32 development board, BMP388 pressure sensor, SHT30 temperature/humidity sensor, BH1750 light sensor, analog rain sensor, SSD1306-type OLED display, active buzzer module, breadboard, power supply, and interconnection wires.

Software and development environments include STM32CubeMX, Keil MDK-ARM, C/C++ with STM32 HAL, Visual Studio Code, PlatformIO, Arduino framework for the ESP32 gateway, MQTT communication, HiveMQ WebSocket Client, Node-RED, and FlowFuse Dashboard.

1.5 SCOPE OF THE PROJECT

The scope of this project is a small-scale prototype weather station for indoor or protected experimental use. The system measures environmental parameters, displays local data, generates a rain warning, and provides remote monitoring through an MQTT dashboard. The design does not include a calibrated industrial enclosure, long-term battery operation, professional weather-proofing, wind speed measurement, cloud database storage, or certified meteorological accuracy.

1.6 RESEARCH METHODOLOGY

1. Review datasheets and documentation for the selected microcontrollers, sensors, and communication protocols.
2. Configure and test STM32 peripherals individually, including clock, GPIO, I2C, ADC, and UART.
3. Integrate each sensor and verify its measured output before combining all modules.
4. Implement local OLED display and rain-state buzzer behavior.
5. Define a compact UART data frame and verify communication between STM32 and ESP32.
6. Implement Wi-Fi and MQTT functions on ESP32 using PlatformIO.
7. Create Node-RED MQTT subscriptions and dashboard widgets.
8. Perform functional, reconnection, and power-cycle tests on the integrated system.

1.7 REPORT ORGANIZATION

Chapter 1 introduces the project background, motivation, objectives, scope, and methodology. Chapter 2 presents the hardware, communication protocols, and development tools used in the system. Chapter 3 describes the overall design, data flow, pin connections, rain classification, UART data frame, MQTT topics, and system flowchart. Chapter 4 explains the hardware and software implementation. Chapter 5 presents experimental results and system evaluation. Chapter 6 summarizes the project and proposes future improvements.

CHAPTER 2: LITERATURE REVIEW

2.1 OVERVIEW OF IoT WEATHER MONITORING SYSTEMS

A weather monitoring system is an embedded system designed to collect environmental data through sensors, process the measured values, and present the results to users. Depending on the system's requirements, the measured parameters may include temperature, relative humidity, atmospheric pressure, light intensity, and rainfall conditions. By combining sensor acquisition with wireless communication, an IoT-based weather station can provide both local monitoring and remote access to environmental information.

In this project, the weather station is built around two microcontrollers: the STM32F103C8T6 and the ESP32. The STM32F103C8T6 is responsible for collecting data from the sensors, processing the measurements, and displaying information on an OLED screen. The ESP32 receives measurement data from the STM32 through UART communication, connects to a Wi-Fi network, and transmits the data to an MQTT broker. Node-RED is used to visualize the collected information, while HiveMQ provides the MQTT broker service for message communication.

The system consists of three main parts: the sensing unit, the processing and local display unit, and the wireless communication and monitoring unit. The sensing unit measures environmental conditions using the SHT30 temperature and humidity sensor, BMP388 atmospheric pressure sensor, BH1750 light sensor, and analog rain sensor. The STM32 processes the readings and updates the OLED display, while the ESP32 handles wireless data transmission. This modular structure separates measurement tasks from network communication, making the system easier to develop, test, and maintain.

2.2 STM32F103C8T6 MICROCONTROLLER

The STM32F103C8T6 is a 32-bit microcontroller based on the ARM Cortex-M3 processor core. It provides integrated peripherals, including GPIO, ADC, I2C, USART, timers, and interrupt controllers, making it suitable for embedded measurement and control applications.

In this project, the STM32F103C8T6 serves as the main controller for sensor acquisition, measurement processing, and local data display. It communicates with the SHT30 temperature and humidity sensor, BMP388 atmospheric pressure sensor, BH1750 light sensor, and SSD1306 OLED display through the I2C interface. The analog rain sensor is connected to the STM32's ADC input, allowing the controller to convert the sensor's output voltage into a digital value for further processing.

After acquiring the sensor readings, the STM32 processes the measurements, updates the OLED display, and organizes the data into a predefined binary communication packet. The packet uses the header bytes 0xAA 0x55, has a total length of 16 bytes, and includes an XOR checksum for basic data-integrity checking. The STM32 then transmits the packet to the ESP32 through the UART interface. The ESP32 checks the received packet and decodes the measurement values before sending them to the MQTT broker.

Separating sensor acquisition and local display from wireless communication reduces the networking workload of the STM32. It also allows the STM32 to continue performing its measurement and display functions independently of the ESP32's Wi-Fi and MQTT communication

status. This architecture improves the modularity of the system and simplifies the testing and maintenance of individual components.

Parameter	Specification
Core	ARM Cortex-M3
Maximum system clock	72 MHz
Operating voltage	3.3 V
Flash memory	64 KB nominal device class
SRAM	20 KB
Interfaces used in this project	I2C1, ADC1, USART1, GPIO

The system clock is configured using an 8 MHz external clock source and a phase-locked loop (PLL) to achieve a 72 MHz system clock. The APB1 peripheral bus operates at 36 MHz, while APB2 operates at 72 MHz. This configuration provides sufficient processing performance for sensor acquisition, data formatting, OLED updates, and UART communication.

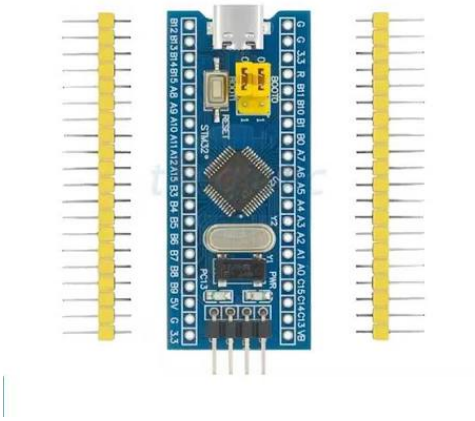


Figure 2.1: STM32F103C8T6

2.3 BMP388 PRESSURE SENSOR

The BME280 is an environmental sensor that integrates temperature, relative humidity, and atmospheric pressure measurements into a single device. It supports I2C and SPI communication. In this project, I2C is selected to simplify the hardware connections and allow several compatible modules to share the same communication bus.

The BME280 provides digital measurement values that can be processed by the STM32. Temperature is expressed in degrees Celsius (°C), relative humidity in percent (%RH), and atmospheric pressure in hectopascals (hPa).

The sensor is used to monitor changes in the surrounding environment. The measured values can be displayed locally on the OLED screen and transmitted to the ESP32 for remote monitoring.

During implementation, the firmware must check whether the sensor initializes successfully and returns valid data. If communication fails, the program should identify the error rather than silently treating an old measurement as a new reading.

The sensor should also be placed away from heat-producing components and positioned where air can circulate freely. These precautions help reduce measurement errors caused by the surrounding hardware.

Item	Project Use
Interface	I2C
Possible I2C addresses	0x76 / 0x77
Measured quantity	Atmospheric pressure
Displayed unit	hPa
STM32 bus	I2C1 on PB6/PB7



Figure 2.2: BMP388 Senor

2.4 SHT30 TEMPERATURE AND HUMIDITY SENSOR

The SHT30 is a digital sensor designed to measure ambient temperature and relative humidity. It communicates with the STM32F103C8T6 through the I2C interface, allowing the microcontroller to retrieve measurement data using a simple digital communication protocol.

In this project, the SHT30 provides the temperature and humidity values used by the weather station. After receiving the measurements, the STM32 processes the readings and converts them into fixed-point values for consistent data formatting. The processed values are then included in a JSON message and transmitted to the ESP32 through UART for wireless communication and remote monitoring.

Using the SHT30 enables the system to obtain temperature and humidity measurements digitally, reducing the need for additional analog signal conditioning.

Item	Project Use
Interface	I2C
Default address	0x44
Measured quantities	Temperature and relative humidity
Temperature unit	°C
Humidity unit	%RH



Figure 2.3: SHT30 Senor

2.5 BH1750 LIGHT SENSOR

The BH1750 is a digital ambient-light sensor used to measure light intensity in lux (lx). It communicates with the STM32F103C8T6 through the I2C interface, allowing the microcontroller to obtain light measurements directly without requiring an external analog-to-digital conversion process.

In this project, the BH1750 sensor is integrated into the weather station to monitor changes in ambient light intensity. The sensor operates in continuous high-resolution mode, providing updated measurements for the STM32 to read and process. The measured light intensity is then included in the system's data message and transmitted to the ESP32 through UART for remote monitoring via the IoT platform.

Item	Project Use
Interface	I2C
Address used	0x23
Output	Ambient light intensity
Unit	Lux
STM32 bus	I2C1

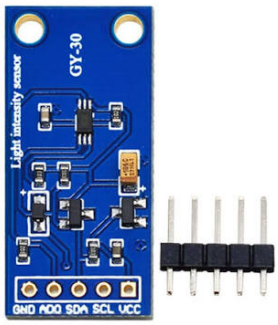


Figure 2.4: BH1750 Sensor

2.6 ANALOG RAIN SENSOR

The analog rain sensor detects the presence of water through changes in electrical conductivity across its exposed sensing plate. As the plate becomes wetter, its electrical characteristics change, causing the sensor's analog output voltage to vary. In this project, the analog output is connected to an ADC input of the STM32F103C8T6, which converts the voltage into a digital value for further processing.

The measured ADC value generally decreases as the sensing plate becomes wetter. However, the raw value may vary depending on water coverage, sensor condition, and environmental factors. Therefore, the system classifies rainfall conditions using calibrated threshold ranges instead of exact-value comparisons.

State	Typical calibrated value	Decision range
DRY	approximately 4000	ADC ≥ 3000
LOW rain	approximately 2000	1500 ≤ ADC < 3000
HEAVY rain	approximately 1000	ADC < 1500

The thresholds are selected based on practical calibration measurements. Using ranges helps distinguish the three rainfall states and reduces incorrect classification caused by small fluctuations in ADC readings. These thresholds are specific to the prototype and may need adjustment depending on the sensor module and actual operating conditions.

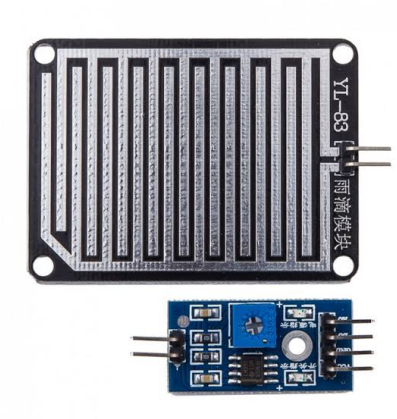


Figure 2.5: Rain sensor

2.7 OLED DISPLAY

A 0.96-inch OLED module is used to display environmental measurements locally. The module uses the SSD1306 display controller and communicates with the STM32F103C8T6 through the I2C interface. It shares the I2C1 bus with the compatible digital sensors, allowing multiple devices to communicate with the microcontroller through the same bus.

In this project, the OLED displays the measured temperature, relative humidity, atmospheric pressure, light intensity, and rainfall status. The STM32 updates the displayed information as new sensor readings become available, enabling users to monitor environmental conditions directly on the device.

The local display provides an additional way to access measurement data without relying on a wireless connection. Therefore, the weather station can continue displaying available sensor readings even when Wi-Fi connectivity or the MQTT service is temporarily unavailable.

Item	Project Use
Interface	I2C
Address	0x3C
Resolution	128 × 64 pixels
Displayed data	T, H, P, L, R / rain state



Figure 2.6: OLED 0.96 Display

2.8 ACTIVE BUZZER

An active buzzer is used to provide audible notifications when the rain sensor detects water. Unlike a passive buzzer, an active buzzer contains an internal oscillation circuit and can generate sound when a suitable control signal is applied. In this project, the buzzer module is active-low, meaning that it is activated by a LOW logic level and deactivated by a HIGH logic level.

The buzzer behavior depends on the rainfall state determined by the STM32F103C8T6. When the sensor indicates the DRY state, the buzzer remains off. In the LOW rain state, the buzzer produces a short beep during each measurement cycle to indicate light rainfall. When the sensor detects the HEAVY rain state, the buzzer remains continuously active to provide a more noticeable warning.

This audible notification allows users to recognize changes in rainfall conditions without continuously checking the OLED display. Combined with the visual display and remote monitoring functions, the buzzer improves the usability of the weather station.

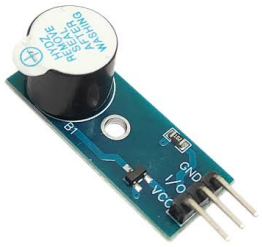


Figure 2.7: Active Buzzer

2.9 ESP32 Wi-Fi GATEWAY

The ESP32 serves as the wireless communication gateway between the STM32F103C8T6 and the IoT platform. Rather than performing the main sensor acquisition tasks, it receives weather data from the STM32 through the UART2 interface. The received data is organized into a binary packet containing the measured values and a checksum for frame validation.

After receiving a packet, the ESP32 verifies its checksum to detect transmission errors and decodes the weather measurements. It then connects to a Wi-Fi network and publishes the processed data to an MQTT broker, allowing the measurements to be accessed by remote monitoring applications.

Separating sensor acquisition from wireless communication allows the STM32 to focus on sensor processing and local display functions, while the ESP32 manages Wi-Fi connectivity and MQTT communication. This architecture helps maintain local system operation even when network connectivity is temporarily unavailable.

Parameter	Project Configuration
Board	ESP32 DOIT DevKit V1
Framework	Arduino through PlatformIO
UART2 RX	GPIO16
UART2 TX	GPIO17
UART baud rate	115200 bit/s
Network role	Wi-Fi and MQTT gateway



Figure 2.8: ESP 32

2.10 COMMUNICATION INTERFACES

2.10.1 I2C

I2C is used for the BMP388, SHT30, BH1750, and OLED display. All devices share the same clock and data lines because each device has a different address. PB6 is configured as I2C1_SCL and PB7 as I2C1_SDA at 100 kHz.

2.10.2 ADC

ADC1 channel 0 on PA0 is used for the analog rain sensor. The STM32 converts the sensor voltage into a 12-bit digital value and then classifies the result into one of three rain states.

2.10.3 UART

USART1 is configured at 115200 bit/s with 8 data bits, no parity, and one stop bit. PA9 is STM32 TX and PA10 is STM32 RX. The primary data direction in the project is STM32 to ESP32, where ESP32 GPIO16 is used as UART2 RX.

2.11 MQTT, HIVEMQ AND NODE-RED

MQTT is a lightweight client-server publish/subscribe messaging protocol designed for machine-to-machine and IoT communication. In this project, the ESP32 acts as an MQTT publisher. HiveMQ is used as the broker, and Node-RED acts as a subscriber and visualization application. The publish/subscribe model allows the ESP32 and dashboard to remain independent while exchanging data through named topics.

Node-RED provides a visual flow environment. The MQTT input node subscribes to all weather topics, a switch node separates messages according to msg.topic, and dashboard nodes display the resulting values as gauges, text fields, and a time-series chart.

2.12 SOFTWARE DEVELOPMENT ENVIRONMENT

Tool	Function
STM32CubeMX	Peripheral, clock, ADC, I2C, UART and GPIO configuration
Keil MDK-ARM	Build, flash and debug STM32 firmware
Visual Studio Code	ESP32 source code editor
PlatformIO	ESP32 project management and upload
HiveMQ WebSocket Client	Manual MQTT subscription and message verification
Node-RED	MQTT data flow and dashboard implementation

Using separate development environments reflects the two-controller architecture. STM32 firmware is developed with the STM32 HAL ecosystem, while the ESP32 gateway is developed using PlatformIO and the Arduino framework.

2.13 CHAPTER SUMMARY

This chapter reviewed the components and technologies used in the weather station. The next chapter describes how these components are combined into the final system architecture and how data move from the sensors to the dashboard.

CHAPTER 3: SYSTEM DESIGN

3.1 SYSTEM REQUIREMENTS

The weather station is designed to provide continuous environmental monitoring through local sensing, data processing, warning, and remote communication. The system requirements are divided into sensing, local operation, communication, and remote visualization.

The sensing function must acquire temperature and relative humidity from the SHT30, atmospheric pressure from the BMP388, light intensity from the BH1750, and an analog reading from the rain sensor. The STM32F103C8T6 processes these measurements and updates the local OLED display. It must also classify the rain condition into DRY, LOW, and HEAVY states and control the active buzzer according to the defined warning logic.

For remote monitoring, the STM32 must transmit the weather measurements to the ESP32 through UART using the predefined binary packet format. The ESP32 must validate incoming packets, maintain Wi-Fi and MQTT connectivity, and publish the measurements to the corresponding MQTT topics. Node-RED must receive these messages and present the environmental values and system status through the dashboard.

The integrated system is also expected to recover from temporary network disconnections and normal power interruptions without requiring manual reconfiguration. These requirements provide the basis for the implementation and functional tests described in the following chapters.

3.2 OVERALL SYSTEM ARCHITECTURE

The system adopts a two-controller architecture to separate environmental data processing from wireless communication. The STM32F103C8T6 acts as the main controller, while the ESP32 operates as the communication gateway between the embedded system and the IoT monitoring platform.

The sensing layer consists of the SHT30, BMP388, BH1750, and analog rain sensor. These devices provide the environmental measurements required by the application. The STM32 acquires the sensor readings, processes the values, classifies the rain condition, and updates the OLED display. It also controls the active buzzer to provide a local warning when the detected rain state requires an audible indication.

The communication layer is implemented by the ESP32. Weather data are transferred from the STM32 through UART in a fixed-length binary packet. After validating and decoding the received packet, the ESP32 publishes the individual measurements to an MQTT broker through Wi-Fi. HiveMQ provides the broker service, and Node-RED subscribes to the relevant topics to display the received information.

This division of responsibilities allows local sensing, display, and warning functions to remain separate from network-dependent services. It also simplifies software development because sensor drivers and local control logic can be developed independently of Wi-Fi, MQTT, and dashboard functions.

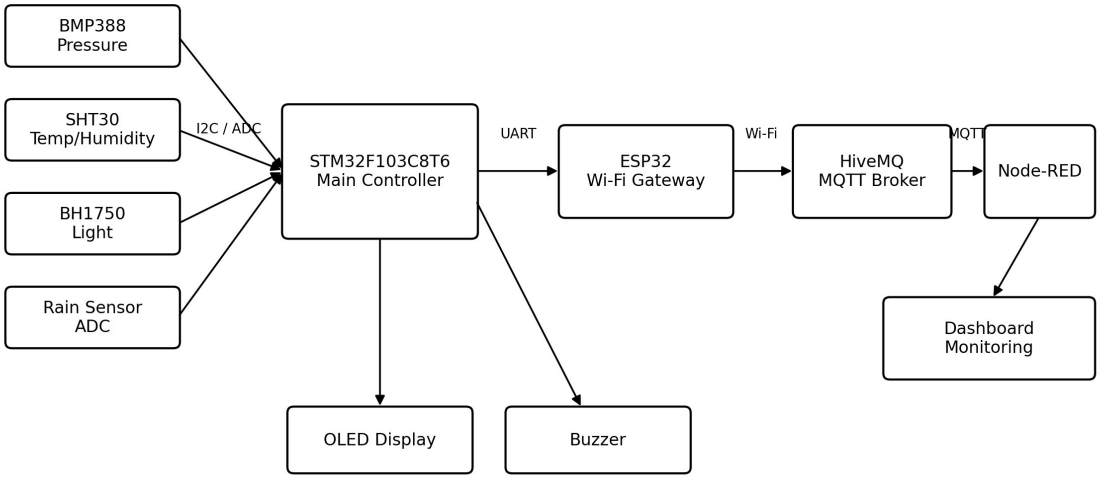


Figure 3.1: Overall block diagram of the proposed weather station

3.3 HARDWARE CONNECTION DESIGN

Module	Module Pin	STM32/ESP32 Pin	Function
BMP388	SCK/SCL	PB6	I2C1 SCL
BMP388	SDI/SDA	PB7	I2C1 SDA
SHT30	SCL	PB6	I2C1 SCL
SHT30	SDA	PB7	I2C1 SDA
BH1750	SCL	PB6	I2C1 SCL
BH1750	SDA	PB7	I2C1 SDA
OLED	SCL	PB6	I2C1 SCL
OLED	SDA	PB7	I2C1 SDA
Rain sensor	AO	PA0	ADC1 channel 0
Buzzer	Signal	PB0	GPIO output, active-low
ESP32	GPIO16 RX	PA9 TX	STM32 USART1 TX
ESP32	GPIO17 TX	PA10 RX	STM32 USART1 RX
All modules	GND	GND	Common ground

All I2C modules share PB6 and PB7. This is possible because their addresses are different. The rain sensor is separated from the I2C bus because it provides an analog voltage. A common ground is required between the STM32 and ESP32 so that UART logic levels have the same reference.

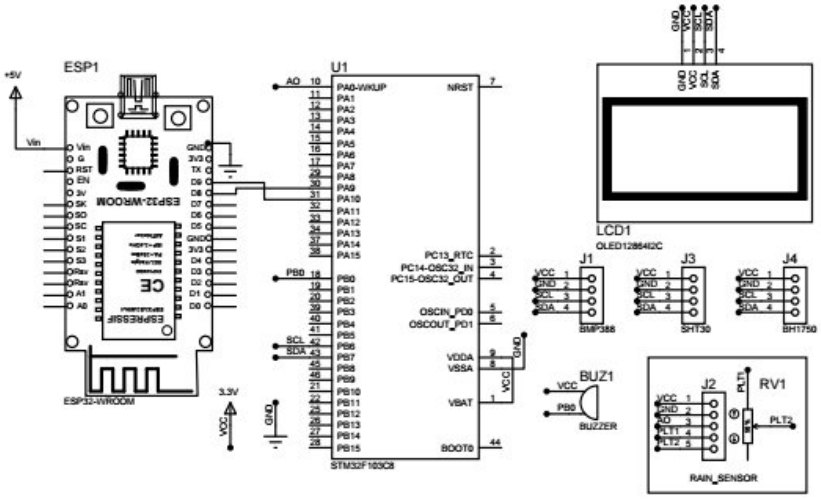


Figure 3.2: Circuit Schematic of the Weather Station System

3.4 SENSOR DATA ACQUISITION AND PROCESSING

The STM32 is responsible for acquiring measurements from all sensors during each measurement cycle. The SHT30 provides temperature and relative humidity, the BMP388 provides atmospheric pressure, and the BH1750 measures ambient light intensity. The analog rain sensor is sampled through the STM32 ADC to obtain a raw digital value representing the sensor output.

After acquisition, the firmware organizes the measurements into a common weather data structure. Sensor values that contain fractional parts are converted into scaled integers before UART transmission. In the current data format, temperature, humidity, and pressure are multiplied by 100, while light intensity is represented as an integer value in lux. This representation preserves two decimal places for the scaled measurements without requiring floating-point values to be transmitted directly.

The rain sensor's raw ADC value is retained for classification and remote monitoring. The firmware uses this value to determine the current rain state according to the configured thresholds. The processed measurements and rain state are then used by the local display, warning logic, and UART communication function.

Keeping data acquisition and processing within the STM32 provides a consistent set of measurements for both local and remote use. The same processed data structure supplies the OLED display and the UART packet, reducing the need to implement separate measurement logic for different output interfaces.

3.5 RAIN CLASSIFICATION AND BUZZER LOGIC

The analog rain sensor produces a voltage that is converted into a digital value by ADC1 channel 0. The firmware uses the resulting raw ADC reading to classify the detected condition into three states: DRY, LOW, and HEAVY. The classification thresholds are selected for the specific sensor used in the prototype and should be verified experimentally before deployment.

Rain State	ADC Condition	Buzzer Behavior
DRY	rain_raw ≥ 3000	OFF
LOW	1500 ≤ rain_raw < 3000	Short 150 ms beep per cycle
HEAVY	rain_raw < 1500	Continuous ON

Simplified rain classification logic

```
if (rain_raw >= 3000)      rain_state = DRY;
else if (rain_raw >= 1500) rain_state = LOW;
else                       rain_state = HEAVY;
```

When the measured value is at or above 3000, the system classifies the condition as DRY and keeps the buzzer off. When the value falls between 1500 and 2999, the condition is classified as LOW and the buzzer produces a short beep during the warning cycle. When the value is below 1500, the condition is classified as HEAVY and the buzzer remains on according to the implemented control logic.

This threshold-based method is straightforward to implement and requires little processing overhead. However, the ADC thresholds depend on the sensor module, supply conditions, and physical installation. They should therefore be treated as prototype-specific calibration values rather than universal rainfall measurements.

3.6 UART WEATHER DATA FRAME

UART is used to transfer the processed weather measurements from the STM32 to the ESP32. To avoid the ambiguity associated with parsing formatted text, the system uses a fixed-length binary packet. Each packet contains a two-byte header, the encoded sensor values, the rain state, and a checksum.

Byte(s)	Type / Value	Meaning
0	0xAA	Header 1
1	0x55	Header 2
2-3	int16	Temperature × 100
4-5	uint16	Humidity × 100
6-9	uint32	Pressure × 100
10-11	uint16	Light intensity (lux)
12-13	uint16	Rain raw ADC
14	uint8	Rain state: 0=DRY, 1=LOW, 2=HEAVY
15	uint8	XOR checksum of bytes 0-14

The packet begins with the header bytes 0xAA and 0x55. The receiver uses this sequence to identify the beginning of a frame. The measurement fields are transmitted in little-endian order, meaning that the least significant byte of each multi-byte value is transmitted first. Temperature, humidity, and pressure use scaled integer representations, while light intensity and the raw rain ADC reading are transmitted as unsigned integer values.

The final byte contains the XOR checksum calculated from bytes 0 through 14. After receiving a complete frame, the ESP32 calculates the checksum again and compares it with the received checksum. A frame is accepted only when the values match; otherwise, the frame is rejected and is not used for MQTT publication.

The fixed packet length simplifies reception and decoding on the ESP32. The header supports frame alignment, while the checksum provides a basic method for detecting transmission errors. This mechanism is suitable for the short UART connection used in the prototype, although it is not intended to provide cryptographic protection or guarantee detection of every possible communication error.

3.7 ESP32 Wi-Fi AND MQTT GATEWAY

The ESP32 serves as the interface between the local embedded system and the remote monitoring platform. Its main responsibilities are receiving UART packets, maintaining Wi-Fi connectivity, managing the MQTT client, and publishing decoded weather measurements.

At startup, the ESP32 initializes UART2 and begins receiving binary data from the STM32. When a complete packet is received, the firmware validates the checksum and decodes the sensor values. Invalid packets are discarded so that corrupted data are not forwarded to the MQTT broker.

The ESP32 then maintains the Wi-Fi connection and establishes an MQTT connection when network access is available. After a valid weather packet has been decoded, the gateway publishes the corresponding values to their assigned topics. If Wi-Fi or the MQTT connection is interrupted, the firmware attempts to restore the connection according to its reconnection logic.

This gateway-based design prevents the STM32 from having to manage network protocols directly. The STM32 can continue its local sensing and warning tasks, while the ESP32 handles the additional processing required for wireless communication and remote data delivery.

3.8 MQTT TOPIC ORGANIZATION

Topic	Payload Example
hcmute/weatherstation/weather-C022C8D8CBB0/temperature	29.18
hcmute/weatherstation/weather-C022C8D8CBB0/humidity	54.78
hcmute/weatherstation/weather-C022C8D8CBB0/pressure	1007.85
hcmute/weatherstation/weather-C022C8D8CBB0/light	75
hcmute/weatherstation/weather-C022C8D8CBB0/rain/raw	4079
hcmute/weatherstation/weather-C022C8D8CBB0/rain/state	DRY
hcmute/weatherstation/weather-C022C8D8CBB0/status	ONLINE

MQTT topics provide a structured method for distributing the weather measurements to remote applications. Instead of sending all measurements through a single dashboard message, the ESP32 publishes individual values to separate topics. This allows Node-RED to subscribe to the required data and route each message to the appropriate dashboard component.

The topic hierarchy uses a common weather-station root followed by the name of the measured parameter. The published values include temperature, humidity, pressure, light intensity, the raw rain ADC reading, the classified rain state, and device status. Each topic has a corresponding payload that represents the latest value or state.

The use of separate topics simplifies message routing and makes the dashboard easier to maintain. It also provides a consistent naming structure that can be extended if additional parameters or weather stations are introduced in the future. The topic names and payload formats must remain consistent between the ESP32 publishing code and the Node-RED subscriptions.

3.9 NODE-RED DASHBOARD DESIGN

Node-RED is used to receive MQTT messages and present the weather measurements through a browser-based dashboard. The MQTT input node subscribes to the weather-station topic hierarchy using the wildcard subscription configured for the project. Incoming messages are routed according to their topic names so that each measurement can be displayed independently.

The dashboard presents temperature, relative humidity, atmospheric pressure, and light intensity using gauge widgets. Additional text fields display the raw rain sensor reading, the classified rain state, and the device status. A time-series chart presents temperature changes over time, allowing users to observe recent variations rather than relying only on the latest measurement.

The dashboard is divided into Environment, Weather Status, and History groups to keep the information organized. This layout separates continuously measured environmental values from discrete warning states and historical trends. Node-RED therefore provides the user-facing monitoring layer, while the ESP32 and MQTT broker handle data delivery.

3.10 SYSTEM OPERATING FLOWCHART

The system begins by initializing the STM32 peripherals and sensor interfaces. After successful initialization, the STM32 repeatedly acquires sensor measurements, processes the values, classifies the rain condition, updates the OLED display, and controls the buzzer. The resulting weather data are then encoded into a binary UART packet and transmitted to the ESP32.

On the communication side, the ESP32 receives the packet and checks its header and checksum before decoding the measurements. Valid data are prepared for MQTT publication, while invalid frames are rejected. The ESP32 also maintains Wi-Fi and MQTT connectivity so that the latest valid measurements can be published when the network and broker are available.

Node-RED receives the published messages and updates the corresponding dashboard widgets. The process then continues with the next measurement cycle. This operating sequence combines continuous local monitoring with remote data publication while keeping sensor processing independent of the network connection.

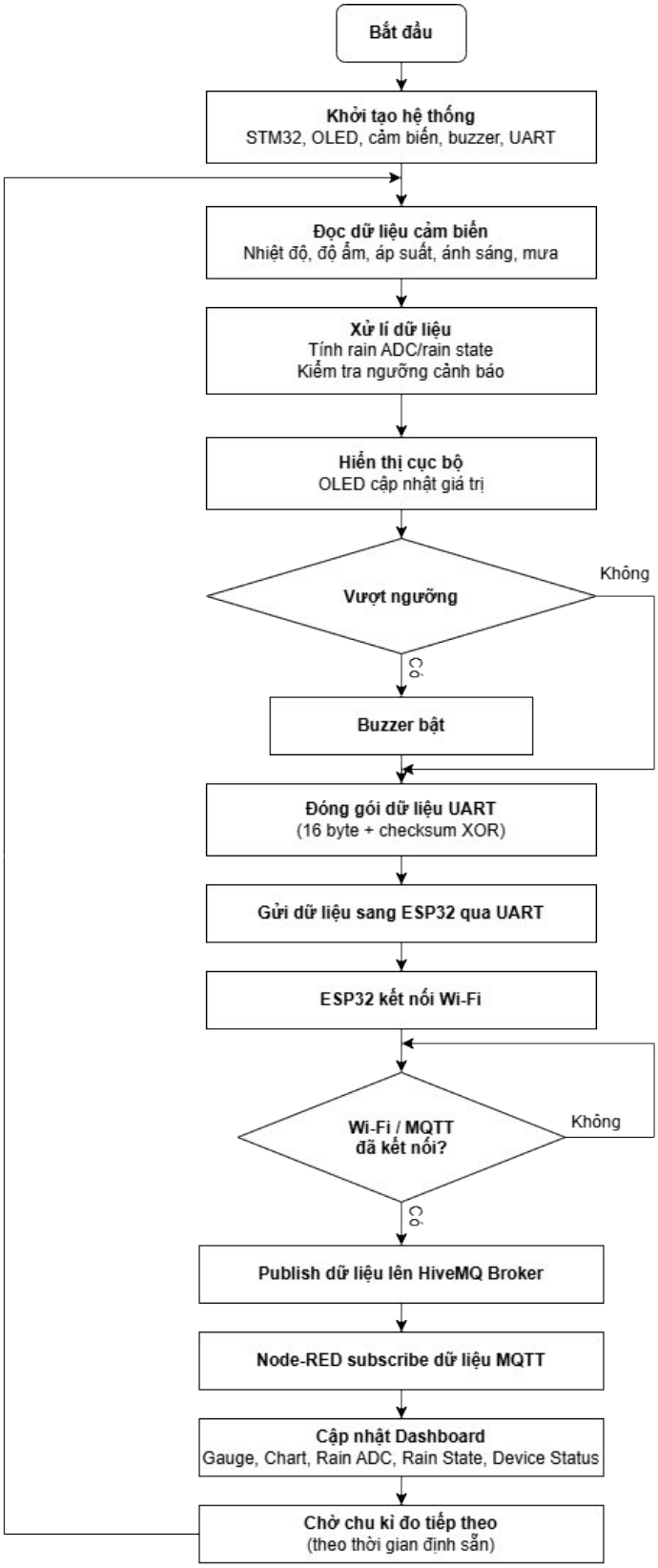


Figure 3.3: System operating flowchart

3.11 SOFTWARE STRUCTURE AND RESPONSIBILITIES

The final software is separated by responsibility instead of placing all device code in one main file. On the STM32 side, dedicated modules handle each sensor and the UART weather packet. The main program coordinates initialization, periodic acquisition, rain classification, local warning, display refresh, and transmission. On the ESP32 side, the program is responsible for frame reception, packet validation, network maintenance, MQTT publication, and serial diagnostics.

Software Module	Main Responsibility
bmp388_stm32	Initialize and read pressure data
sht30_stm32	Read temperature and humidity
bh1750_stm32	Read light intensity in lux
rain_sensor_stm32	Read raw ADC value from the rain sensor
weather_uart	Build and transmit the weather data packet
STM32 main loop	Coordinate sensing, display, warning, and UART transmission
ESP32 main.cpp	Decode UART frame, manage Wi-Fi/MQTT, publish data
Node-RED flow	Subscribe, route by topic, and visualize data

CHAPTER 4: SYSTEM IMPLEMENTATION

4.1 HARDWARE IMPLEMENTATION

The prototype was assembled on a breadboard so that each module could be tested and replaced easily during development. The STM32 board, sensors, OLED, buzzer, and ESP32 were connected according to the pin assignment presented in Chapter 3. The sensing plate of the rain sensor was placed outside the main breadboard area so that water could be applied without wetting the other electronic modules.

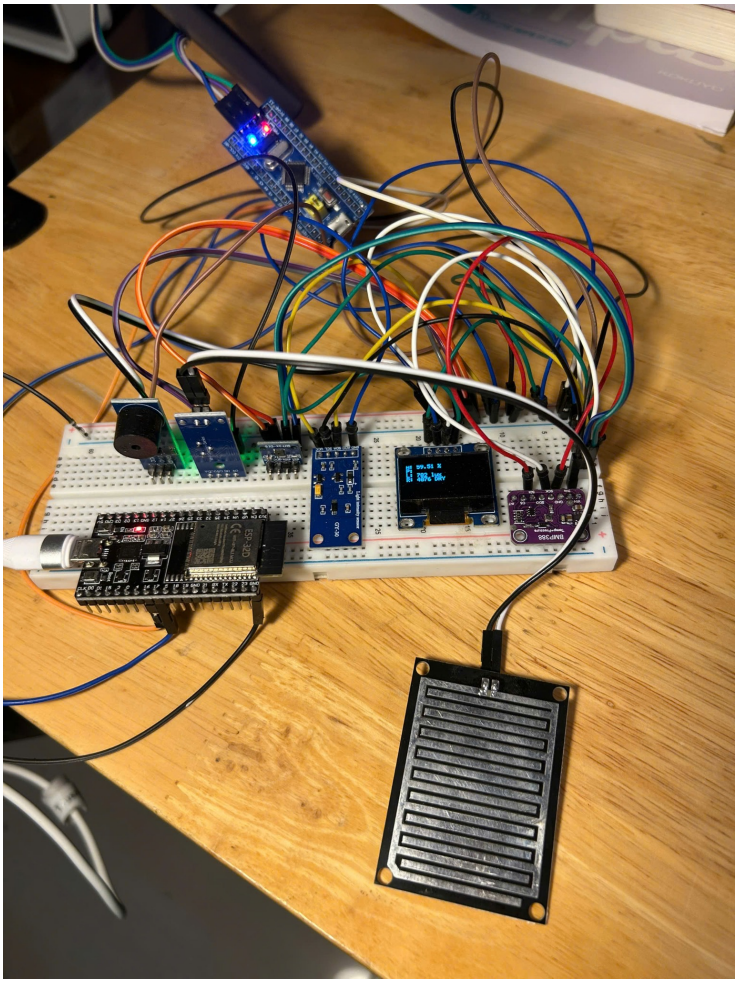


Figure 4.1: Final hardware prototype

The prototype uses jumper wires and development boards rather than a custom PCB. This arrangement is appropriate for functional verification, although a final deployable weather station would require a compact PCB and weather-resistant enclosure.

4.2 STM32 FIRMWARE IMPLEMENTATION

STM32CubeMX was used to configure the clock tree and peripherals. The generated project was compiled in Keil MDK-ARM. The final firmware is organized into separate sensor/interface modules

including bmp388_stm32, sht30_stm32, bh1750_stm32, rain_sensor_stm32, and weather_uart. This modular structure makes the main program easier to read and allows individual devices to be tested separately.

Peripheral	Final Configuration
System clock	72 MHz
I2C1	PB6 SCL, PB7 SDA, 100 kHz
ADC1	Channel 0 on PA0
USART1	PA9 TX, PA10 RX, 115200 8N1
Buzzer GPIO	PB0 output
Status LED	PC13 output

The main loop reads all sensors, performs the rain classification, controls the buzzer, updates the WeatherData_t structure, transmits the UART packet, and refreshes the OLED. A one-second measurement interval is sufficient for the prototype because the environmental parameters change slowly compared with the microcontroller processing speed.

4.3 ESP32 FIRMWARE IMPLEMENTATION

The ESP32 project was developed using Visual Studio Code and PlatformIO with the Arduino framework. HardwareSerial UART2 is configured with GPIO16 as RX and GPIO17 as TX at 115200 bit/s. The receiver is implemented as a small state machine that searches for the two-byte header, receives the complete 16-byte packet, and verifies the XOR checksum before using the data.

After a valid frame is decoded, the ESP32 prints the values to the USB serial monitor, ensures Wi-Fi is connected, ensures the MQTT client is connected, and publishes the latest values. The use of serial output was important during integration because it allowed the UART, Wi-Fi, MQTT, IP address, RSSI, and publish status to be checked at the same time.

Representative ESP32 gateway configuration

```
WeatherSerial.begin(115200, SERIAL_8N1, 16, 17);
WiFi.begin(ssid, password);
mqttClient.setServer("broker.hivemq.com", 1883);
```

4.4 MQTT BROKER IMPLEMENTATION

HiveMQ public broker services were used during development. The ESP32 connects to the broker through MQTT over TCP. For manual verification, the HiveMQ WebSocket Client subscribes to the weather-station root topic with the # wildcard. This provides an independent method to confirm that messages reach the broker before Node-RED is considered.

4.5 NODE-RED FLOW IMPLEMENTATION

The final Node-RED flow is intentionally simple. One MQTT input node subscribes to all weather topics. A switch node separates the messages into seven outputs based on msg.topic. The outputs are directly connected to Dashboard 2.0 widgets. Development debug nodes were removed from the final flow after verification.

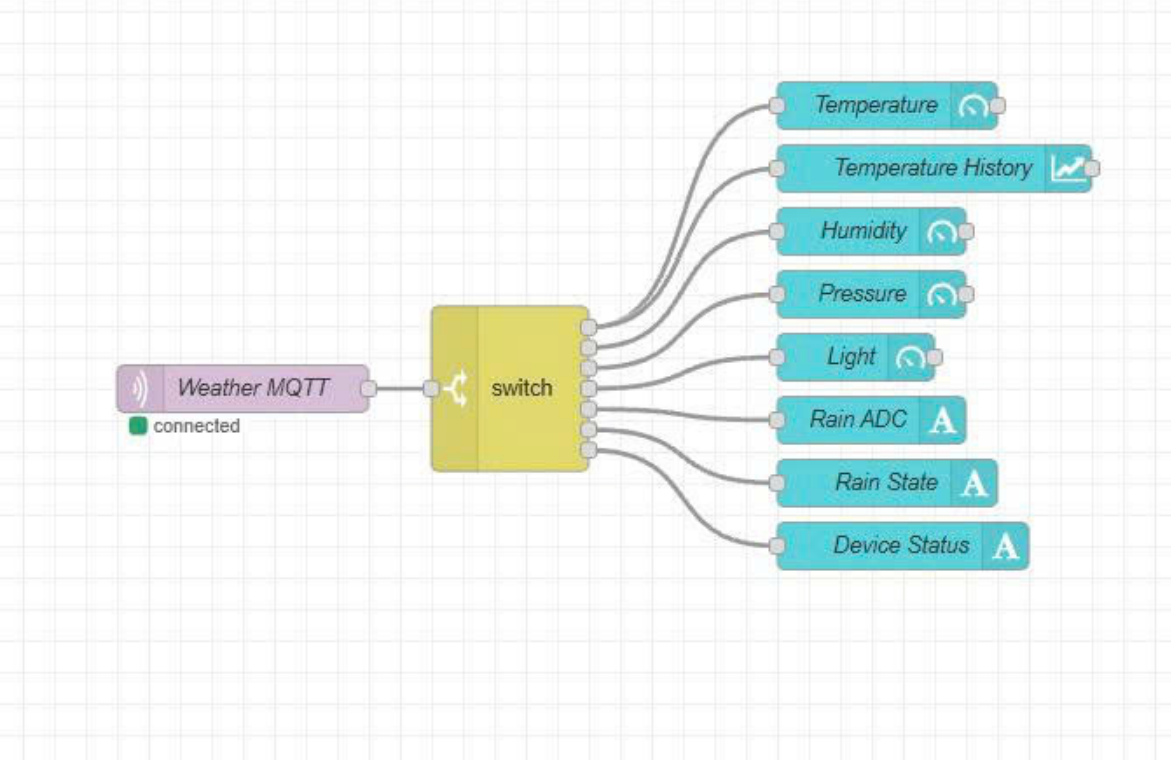


Figure 4.2: Final Node-RED MQTT flow

4.6 DASHBOARD IMPLEMENTATION

The dashboard contains four half-gauge widgets for temperature, humidity, pressure, and light. Text widgets display the raw rain ADC value, classified rain state, and online device status. A time-series chart displays temperature history. The page is divided into Environment, Weather Status, and History groups to keep the interface readable.

The dashboard uses the same MQTT data that are visible through the HiveMQ client, so it serves as the main user-facing visualization while the broker client remains a diagnostic tool.

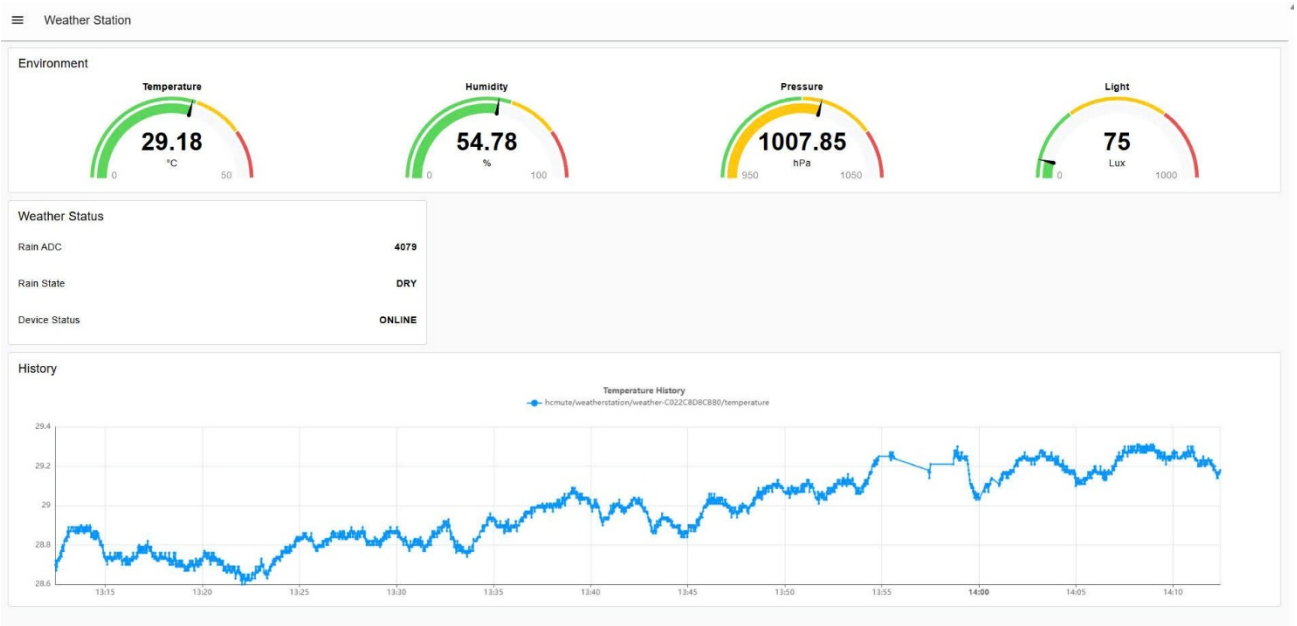


Figure 4.3: Dashboard Implemention

4.7 STARTUP, RECONNECTION, AND FAILURE RECOVERY

The integrated system was designed so that it does not require manual interaction after normal power-up. The STM32 initializes its peripherals and immediately begins local sensing after the sensor checks pass. The ESP32 starts its UART receiver, connects to the configured Wi-Fi network, and then connects to the MQTT broker. If Wi-Fi is unavailable, the network state is reported through the serial monitor and the connection is retried. MQTT reconnection is attempted after Wi-Fi becomes available again.

Condition	System Response
Normal startup	Sensors initialize, OLED updates, UART starts, Wi-Fi and MQTT connect
Wi-Fi lost	ESP32 reports DISCONNECTED and retries Wi-Fi
Wi-Fi restored	ESP32 reacquires an IP address automatically
MQTT lost	ESP32 retries broker connection
MQTT restored	Publication resumes automatically
Power restored	Both controllers restart and the complete data path recovers automatically

This recovery behavior is important because the STM32 local functions continue to operate even when the network is temporarily unavailable. Therefore, loss of Internet connectivity affects remote monitoring but does not stop sensor acquisition, OLED display, or local rain warning.

CHAPTER 5: RESULTS AND SYSTEM EVALUATION

5.1 SYSTEM STARTUP AND SENSOR OPERATION

After power is applied, the STM32 initializes the sensors and local peripherals, while the ESP32 initializes UART communication, connects to Wi-Fi, and connects to the MQTT broker. The complete system starts automatically without requiring manual configuration after each restart.

During testing, the system continuously produced valid temperature, humidity, pressure, light, and rain values. The measurements shown in the following figures are representative readings from the completed prototype rather than fixed demonstration values.

5.2 LOCAL OLED DISPLAY RESULT

The OLED successfully displays all important measurements locally. This local output is independent of the MQTT dashboard and remains useful when the network is unavailable.

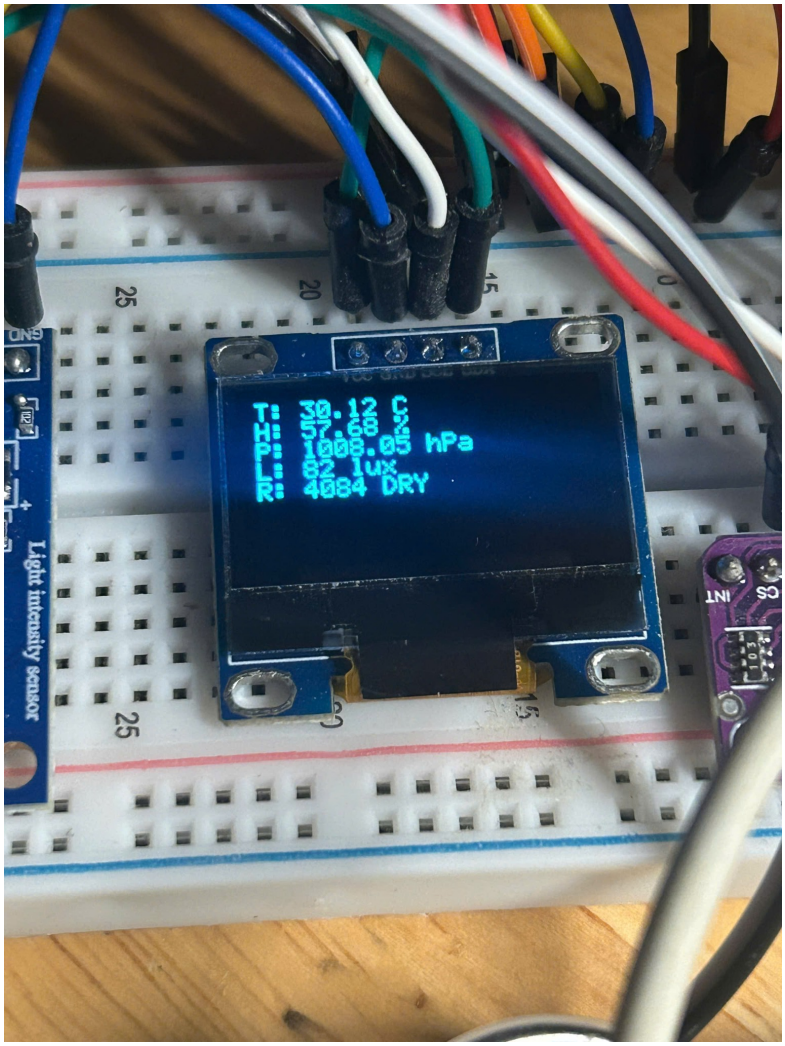


Figure 5.1: OLED display showing weather measurements

5.3 STM32-TO-ESP32, Wi-Fi AND MQTT RESULT

The ESP32 serial monitor confirms that weather packets are received from the STM32 and decoded correctly. The serial output also shows the assigned IP address, RSSI, MQTT connection state, and successful publication status. This verifies the complete STM32-to-ESP32 communication stage before the data reach the broker.

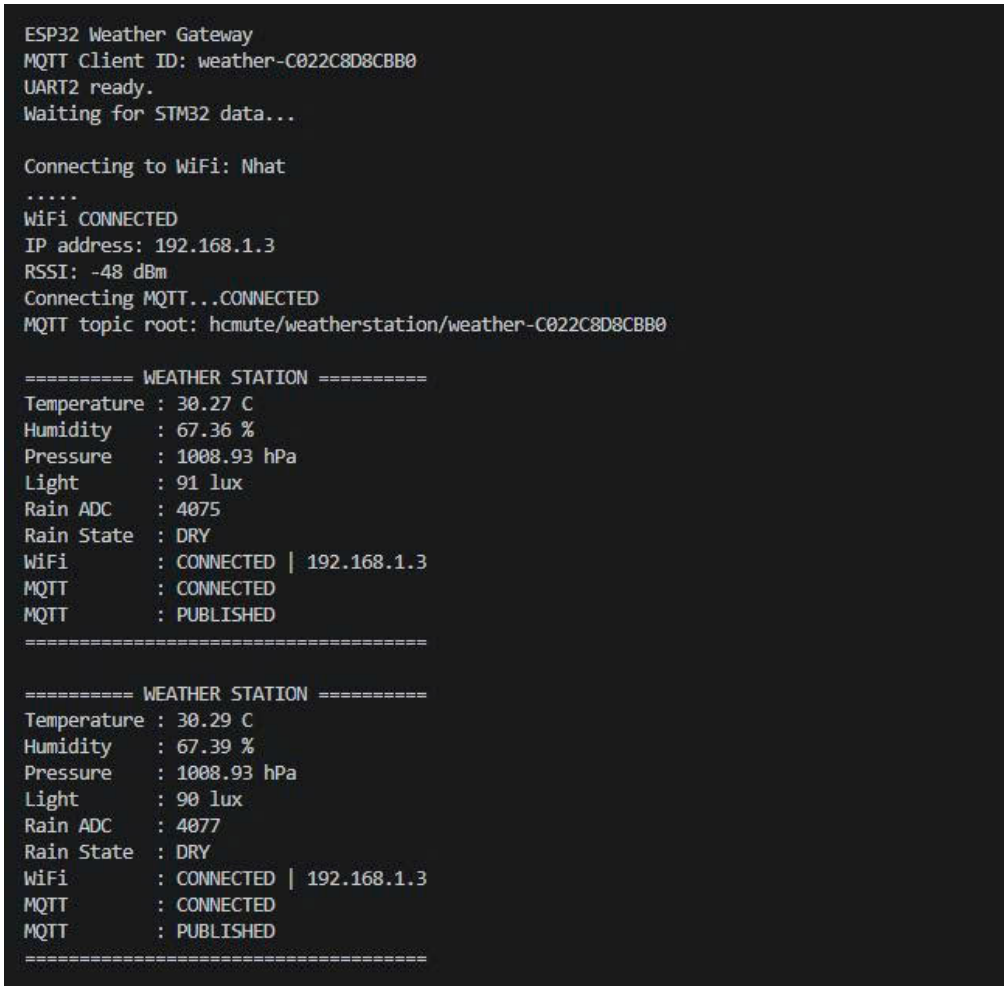


Figure 5.2: ESP32 serial monitor with weather data and network status

5.4 MQTT BROKER RESULT

The HiveMQ WebSocket Client was used to independently subscribe to the weather-station topic. The received values correspond to the values shown in the ESP32 serial monitor, confirming that the ESP32 publishes data successfully over the network.

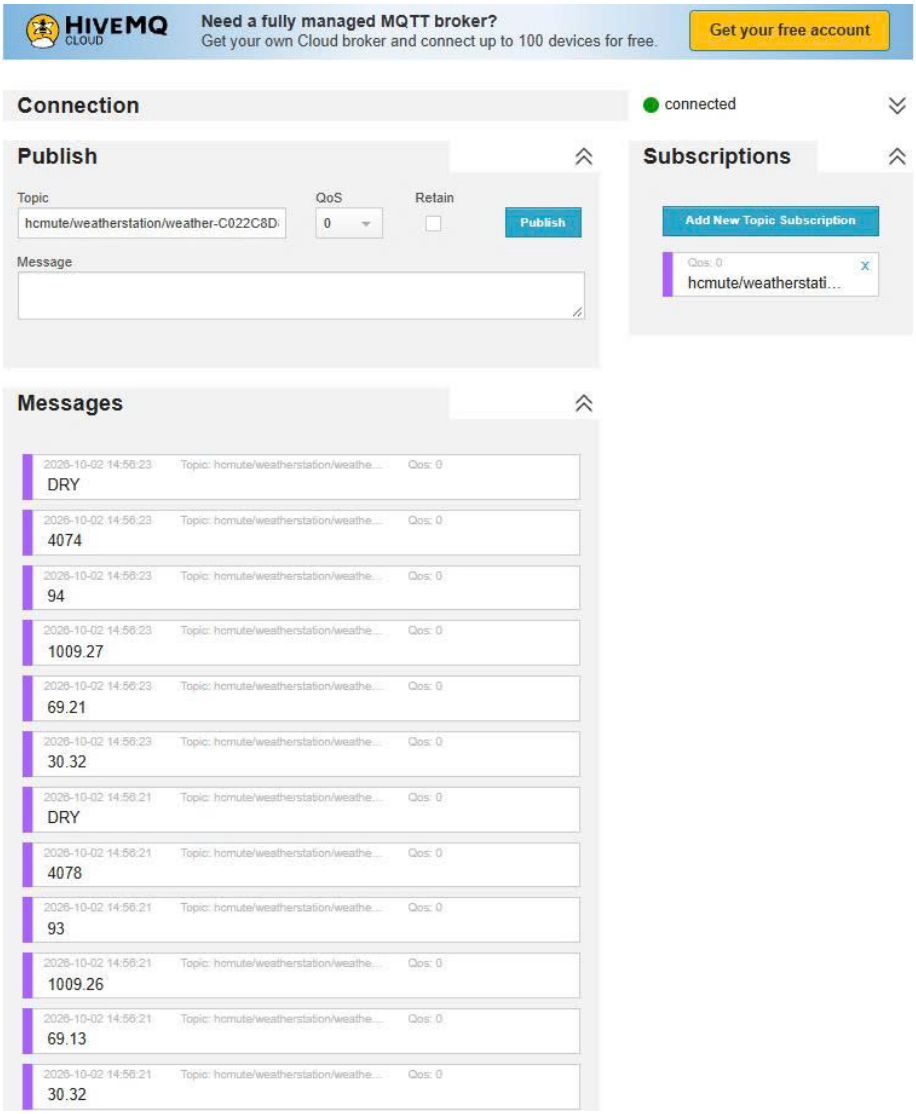


Figure 5.3: Weather messages received through HiveMQ

5.5 NODE-RED DASHBOARD RESULT

The Node-RED dashboard successfully presents the four environmental variables as gauges and the weather status as text. The temperature chart records historical variation, which demonstrates that repeated MQTT messages are received and added to the time-series display.

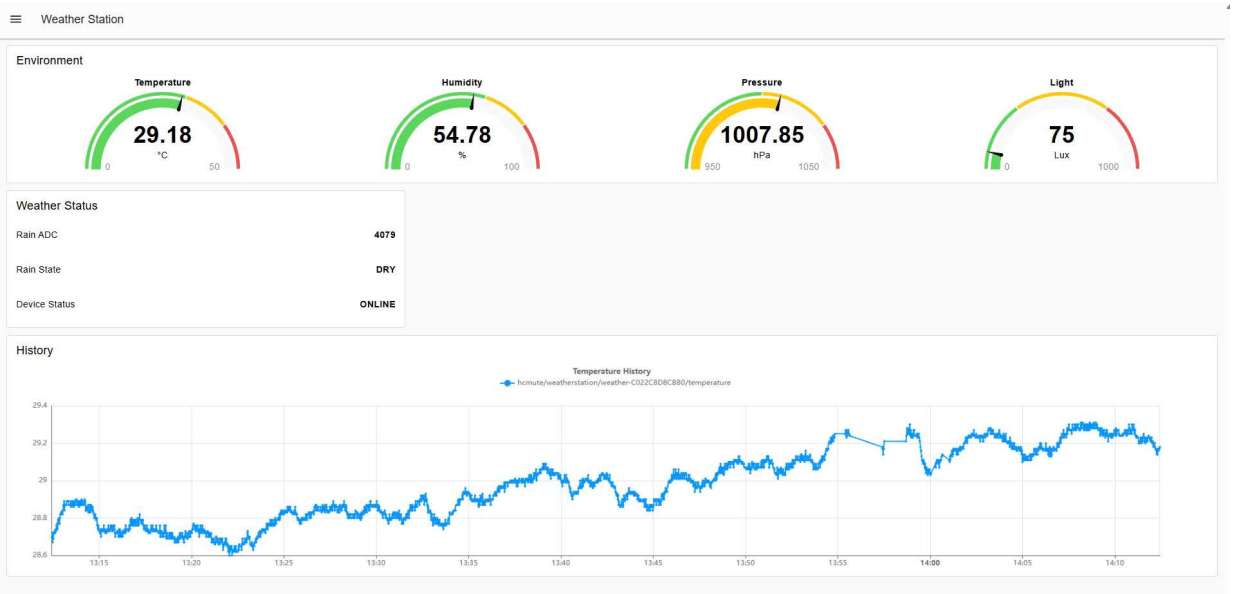


Figure 5.4: Final Node-RED weather dashboard

5.6 RAIN SENSOR AND BUZZER TEST

The rain sensor was tested under dry, partially wet, and strongly wet conditions. The classification thresholds and buzzer behavior were verified experimentally. The state changed from DRY to LOW and HEAVY as the ADC reading decreased.

Test Condition	ADC Range	Expected State	Observed Buzzer	Result
Dry plate	≥ 3000	DRY	OFF	PASS
Partially wet	1500-2999	LOW	Short beep	PASS
Strongly wet	< 1500	HEAVY	Continuous ON	PASS

5.7 Wi-Fi AND MQTT RECONNECTION TEST

To evaluate communication recovery, the Wi-Fi access point was intentionally turned off while the system was operating. The ESP32 reported Wi-Fi and MQTT disconnection. During the outage, MQTT DNS lookup failed as expected because Internet access was unavailable. After Wi-Fi was restored, the ESP32 obtained an IP address again, reconnected to the MQTT broker, and resumed publication without a manual reset.

Step	Observed Behavior	Result
Normal operation	Wi-Fi CONNECTED, MQTT CONNECTED, PUBLISHED	PASS
Wi-Fi disabled	Wi-Fi DISCONNECTED, MQTT DISCONNECTED	PASS
Wi-Fi restored	New IP acquired automatically	PASS
MQTT recovery	MQTT reconnects and publication resumes	PASS

5.8 POWER-CYCLE TEST

The complete STM32 and ESP32 system was powered off and restarted. After power was restored, the sensors initialized, the OLED displayed data, the STM32 resumed UART transmission, the ESP32 reconnected to Wi-Fi and MQTT, and the dashboard began receiving new values. No manual reset or reconfiguration was required. This test confirms that the prototype can recover from normal power interruption.

5.9 SYSTEM EVALUATION

Function	Evaluation
Sensor acquisition	Stable operation for BMP388, SHT30, BH1750 and rain sensor
Local display	All main measurements displayed on OLED
Rain warning	Three states correctly control buzzer behavior
UART communication	Binary packet with header and checksum decoded correctly
Wi-Fi connectivity	Automatic connection and reconnection verified
MQTT	Publish and broker reception verified
Node-RED	Real-time gauges, status fields and chart operate correctly
Restart behavior	Power-cycle recovery verified

5.10 EXISTING LIMITATIONS

- The breadboard prototype is not weather-proof and is intended for laboratory demonstration.
- The rain thresholds are based on practical calibration of the specific sensor and may require adjustment for another sensor or installation.
- The public MQTT broker is appropriate for testing but does not provide the authentication and privacy expected in a production deployment.
- The dashboard stores only the data available in the active Node-RED session; no dedicated long-term database was implemented.
- The system does not include wind speed, wind direction, or rainfall quantity measurement.
- The prototype uses wired USB/power supplies and was not optimized for battery operation.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The STM32 IoT Weather Station was successfully designed, implemented, and tested. The project achieved its main objective of integrating multiple environmental sensors with an STM32F103C8T6 while providing both local and remote monitoring. The STM32 reads the SHT30, BMP388, BH1750, and rain sensor, displays the information on an OLED, classifies rain level, and controls a buzzer warning. The ESP32 receives the processed data through UART and provides Wi-Fi and MQTT connectivity.

Remote monitoring was implemented with HiveMQ and Node-RED. The MQTT topic structure separates each weather parameter, and the Node-RED dashboard presents real-time gauges, rain/device status, and temperature history. Functional testing confirmed sensor operation, OLED display, buzzer warning, STM32-to-ESP32 communication, MQTT publication, dashboard update, automatic Wi-Fi/MQTT reconnection, and power-cycle recovery.

The project therefore demonstrates a complete embedded IoT data path from physical sensors to a web dashboard. The two-controller architecture also makes the system easier to understand and extend because local control and network communication are handled by separate devices.

6.2 FUTURE WORK

- Design a compact PCB and weather-resistant enclosure for outdoor use.
- Add long-term data storage using a local database or cloud platform.
- Use an authenticated/TLS MQTT broker instead of a public test broker.
- Add additional weather parameters such as wind speed, wind direction, or measured rainfall quantity.
- Implement sensor calibration and validation against a reference weather instrument.
- Add mobile-friendly remote access, notifications, and configurable alarm thresholds.
- Optimize power consumption and add battery/solar operation for standalone deployment.

REFERENCES

[1] STMicroelectronics, STM32F103x8/xB Datasheet and Reference Manual, STMicroelectronics.

[2] Bosch Sensortec, BMP388 Digital Pressure Sensor Datasheet and BMP3 Sensor API documentation.

[3] Sensirion, SHT3x-DIS Digital Humidity and Temperature Sensor Datasheet.

[4] ROHM Semiconductor, BH1750FVI Digital Ambient Light Sensor Datasheet.

[5] Espressif Systems, ESP32 Series Datasheet and Arduino-ESP32 documentation.

[6] OASIS, MQTT Version 3.1.1, OASIS Standard, 2014.

[7] Node-RED, Node-RED Documentation and User Guide.

[8] HiveMQ, MQTT Essentials and HiveMQ WebSocket Client documentation.

[9] STMicroelectronics, STM32CubeMX and STM32 HAL documentation.

[10] PlatformIO, PlatformIO Core and Espressif32 Platform documentation.

APPENDIX A: FINAL PIN CONNECTION SUMMARY

Device	Signal	Connection
BMP388	SCK/SCL	PB6
BMP388	SDI/SDA	PB7
SHT30	SCL	PB6
SHT30	SDA	PB7
BH1750	SCL	PB6
BH1750	SDA	PB7
OLED	SCL	PB6
OLED	SDA	PB7
Rain sensor	AO	PA0
Buzzer	Signal	PB0
ESP32	GPIO16 RX	PA9 TX
ESP32	GPIO17 TX	PA10 RX
All modules	GND	GND

The table above records the final prototype wiring used during the successful system tests. Power and ground connections must be shared appropriately, and a common ground between the STM32 and ESP32 is required for UART communication.

APPENDIX B: MQTT TOPIC LIST

Topic	Meaning
hcmute/weatherstation/weather-C022C8D8CBB0/temperature	Temperature in °C
hcmute/weatherstation/weather-C022C8D8CBB0/humidity	Relative humidity in %
hcmute/weatherstation/weather-C022C8D8CBB0/pressure	Pressure in hPa
hcmute/weatherstation/weather-C022C8D8CBB0/light	Light intensity in lux
hcmute/weatherstation/weather-C022C8D8CBB0/rain/raw	Raw rain ADC value
hcmute/weatherstation/weather-C022C8D8CBB0/rain/state	DRY / LOW / HEAVY
hcmute/weatherstation/weather-C022C8D8CBB0/status	ONLINE device status

APPENDIX C: FINAL FUNCTIONAL CHECKLIST

Item	Status
All sensors read correctly	PASS
OLED display updates	PASS
Rain DRY/LOW/HEAVY classification	PASS
Buzzer warning	PASS
STM32 → ESP32 UART	PASS
ESP32 Wi-Fi	PASS
MQTT publish	PASS
HiveMQ subscription	PASS
Node-RED dashboard	PASS
Wi-Fi/MQTT reconnection	PASS
Power-cycle recovery	PASS

APPENDIX D: DEMONSTRATION AND TEST PROCEDURE

The following sequence can be used during final demonstration to verify the complete data path without changing the firmware or dashboard configuration.

Step	Demonstration Action	Expected Result
1	Power on the STM32 and ESP32	OLED starts, sensors are read, ESP32 receives UART data
2	Observe the OLED and compare with ESP32 serial output	Temperature, humidity, pressure, light, and rain values are consistent
3	Open the Node-RED dashboard	Gauges and status fields update in real time
4	Apply water to the rain sensor	Rain state changes from DRY to LOW/HEAVY and buzzer responds
5	Temporarily disable Wi-Fi	ESP32 reports Wi-Fi/MQTT disconnection while STM32 local functions continue
6	Restore Wi-Fi	ESP32 reconnects to Wi-Fi and MQTT and publication resumes automatically
7	Power-cycle the complete system	The full sensing-to-dashboard path recovers without manual reconfiguration

This test sequence verifies both normal operation and recovery behavior. For the final report or presentation, screenshots of the OLED, ESP32 serial monitor, HiveMQ messages, Node-RED flow, and dashboard can be used as supporting evidence.